

Containerization_and_DevOps

Experiment 12: Study and Analyse Container Orchestration using Kubernetes

Objective

Learn why Kubernetes is used, understand its core concepts, and perform deployment, scaling, and self-healing using Kubernetes commands.

Why Kubernetes over Docker Swarm?

- Industry Standard → Widely used in companies
- Powerful Scheduling → Automatically assigns workloads
- Large Ecosystem → Supports many tools and plugins
- Cloud-Native → Works with AWS, GCP, Azure

Core Kubernetes Concepts

- Container → Pod (smallest unit in Kubernetes)
- Compose Service → Deployment (defines how app runs)
- Load Balancing → Service (exposes app)
- Scaling → ReplicaSet (maintains pod count)

Hands-On Lab (k3d / Minikube)

Assumption: kubectl and cluster already installed

```
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS$ kubectl version --client
Client Version: v1.34.1
Kustomize Version: v5.7.1
```

◆ Task 1: Create Deployment

Create `wordpress-deployment.yaml` `apiVersion: apps/v1`

`kind: Deployment`

`metadata:`

`name: wordpress`

`spec:`

`replicas: 2`

`selector:`

`matchLabels:`

`app: wordpress`

`template:`

`metadata:`

`labels:`

`app: wordpress`

`spec:`

`containers:`

`- name: wordpress`

`image: wordpress:latest`

`ports:`

`- containerPort: 80`

```
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS$ nano wordpress-deployment.yaml
```

▶ Apply: `kubectl apply -f wordpress-deployment.yaml`

```
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS$ kubectl apply -f wordpress-deployment.yaml
deployment.apps/wordpress created
```

◆ Task 2: Expose as Service

Create `wordpress-service.yaml` `apiVersion: v1`

`kind: Service`

`metadata:`

`name: wordpress-service`

`spec:`

`type: NodePort`

`selector:`

`app: wordpress`

`ports:`

`- port: 80`

`targetPort: 80`

`nodePort: 30007`

```
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS$ nano wordpress-service.yaml
```

▶ Apply: `kubectl apply -f wordpress-service.yaml`

```
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS$ kubectl apply -f wordpress-service.yaml
service/wordpress-service created
```

◆ Task 3: Verify

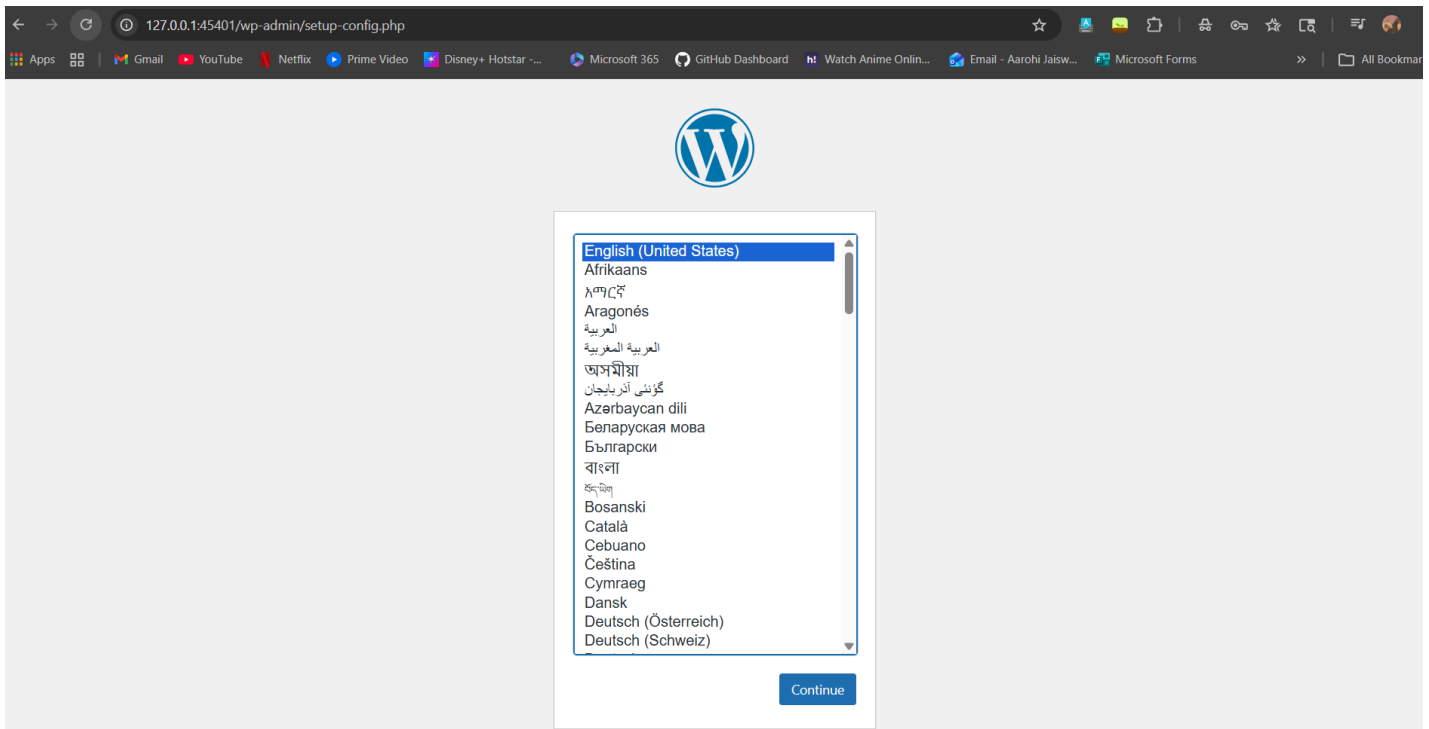
▶ Check pods: `kubectl get pods`

```
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS$ kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
wordpress-6698dd7d66-4t6f2         1/1     Running   0           90s
wordpress-6698dd7d66-jmlkx         1/1     Running   0           90s
```

▶ Check service: `kubectl get svc`

```
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS$ kubectl get svc
NAME                TYPE        CLUSTER-IP   EXTERNAL-IP   PORT(S)          AGE
kubernetes          ClusterIP   10.96.0.1    <none>         443/TCP          10m
wordpress-service    NodePort    10.106.157.6 <none>         80:30007/TCP     6s
```

▶ Access: `http://<node-ip>:30007`



▶ Get IP: `minikube ip`

```
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS$ minikube ip
192.168.49.2
```

◆ Task 4: Scale Deployment

▶ Scale: `kubectl scale deployment wordpress --replicas=4`

```
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS$ kubectl scale deployment wordpress --replicas=4
deployment.apps/wordpress scaled
```

▶ Verify: `kubectl get pods`

```
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS$ kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
wordpress-6698dd7d66-457jz         1/1     Running   0           17s
wordpress-6698dd7d66-4t6f2         1/1     Running   0           11m
wordpress-6698dd7d66-jmlkx         1/1     Running   0           11m
wordpress-6698dd7d66-mc86g         1/1     Running   0           17s
```

◆ Task 5: Self-Healing

▶ Commands: `kubectl get pods`

```
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS$ kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
wordpress-6698dd7d66-457jz         1/1     Running   0           27s
wordpress-6698dd7d66-4t6f2         1/1     Running   0           11m
wordpress-6698dd7d66-jmlkx         1/1     Running   0           11m
wordpress-6698dd7d66-mc86g         1/1     Running   0           27s
```

`kubectl delete pod <pod-name>`

```
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS$ kubectl delete pod wordpress-6698dd7d66-457jz
pod "wordpress-6698dd7d66-457jz" deleted from default namespace
```

`kubectl get pods`

```
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS$ kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
wordpress-6698dd7d66-4t6f2         1/1     Running   0           12m
wordpress-6698dd7d66-jdtxg         1/1     Running   0            8s
wordpress-6698dd7d66-jmlkx         1/1     Running   0           12m
wordpress-6698dd7d66-mc86g         1/1     Running   0           96s
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS$
```

Swarm vs Kubernetes

- Swarm → Easy but limited
- Kubernetes → Advanced and industry standard

Tool Usage

- k3d → Quick learning

- Minikube → Local testing
- kubeadm → Production setup

Cheat Sheet

```
kubectl apply -f file.yaml
kubectl get pods
kubectl get svc
kubectl scale deployment <name> --replicas=N
kubectl delete pod <pod-name>
kubectl get nodes
```

Conclusion

- Learned Kubernetes basics
- Deployed WordPress
- Performed scaling
- Tested self-healing
- Understood real cluster setup

 Next: Deploy your own app